

AIChE CUSC — Backend API Reference

For the Frontend Team

Version: 1.0 · **Base URL:** <http://127.0.0.1:8000> (local) or your deployed domain
Swagger UI: {BASE_URL}/docs · **ReDoc:** {BASE_URL}/redoc

Table of Contents

- [Architecture Overview](#)
- [Authentication & Authorization](#)
- [User Roles & Permissions Matrix](#)
- [Pre-Seeded Admin Accounts](#)
- [API Endpoints Reference](#)
- [5.1 Auth](#)
- [5.2 Dashboard](#)
- [5.3 Courses](#)
- [5.4 Workshops](#)
- [5.5 Events & Site Visits](#)
- [5.6 Committee Sessions](#)
- [5.7 Points](#)
- [5.8 Committee Database](#)
- [5.9 Monthly Reports](#)
- [5.10 Feedback & Moderation](#)
- [5.11 Access Permissions](#)
- [5.12 Certificates](#)
- [Business Logic Rules](#)
- [Point System Scoring Tables](#)
- [Error Handling Guide](#)
- [Frontend Tips & Best Practices](#)

1. Architecture Overview



- **Framework:** FastAPI (Python)
- **Database:** PostgreSQL on Neon (cloud)
- **Auth:** JWT Bearer Tokens (OAuth2)
- **CORS:** Fully open (*) — all origins, methods, headers allowed
- **Static Files:** Served at </static/> (generated certificates)

2. Authentication & Authorization

How Login Works

1. **Send credentials** to `POST /api/auth/login` using `application/x-www-form-urlencoded` (OAuth2 standard)
2. **Receive a JWT token** in the response
3. **Attach the token** to every subsequent request as `Authorization: Bearer <token>`

Token Details

Property	Value
Algorithm	HS256
Expiry	24 hours (1440 min)
Payload Fields	<code>user_id, role, exp</code>

Login Request Format

[!IMPORTANT]

The login endpoint uses **OAuth2 form data**, NOT JSON. The field is called `username` but accepts both username and email.

```
POST /api/auth/login
Content-Type: application/x-www-form-urlencoded

grant_type=password&username=president&password=AICHECUSGeneral%402026
```

Attaching Token to Requests

```
// In your API client (axios, fetch, etc.)
const headers = {
  'Authorization': `Bearer ${accessToken}`,
  'Content-Type': 'application/json'
};
```

3. User Roles & Permissions Matrix

Role Hierarchy (highest -> lowest)

Role	DB Value	Description
President	<code>president</code>	Full system access, sees everything
Vice President	<code>vice_president</code>	Same as President
Secretary	<code>secretary</code>	Same as President
Committee Admin	<code>committee_admin</code>	Head of a specific committee
Member	<code>member</code>	Regular committee member
Student	<code>student</code>	Basic user, signed up via /signup

Permissions by Role

Feature	Student	Member	Committee Admin	President/VP/Secretary
View Courses	Yes	Yes	Yes	Yes
Add/Edit/Delete Courses	No	No	Yes (Technical/Web Dev only)	Yes
View Workshops	Yes	Yes	Yes	Yes
Add/Edit/Delete Workshops	No	No	Yes (own committee)	Yes
View Sessions	No	Yes (own + public)	Yes (own + public; QC sees all)	Yes (all)
Create Sessions	No	No	Yes	Yes
Score Members	No	No	Yes	Yes

Feature	Student	Member	Committee Admin	President/VP/Secretary
View Points	No	Yes	Yes	Yes
View Events	Yes	Yes	Yes	Yes (no auth needed)
Create Events	No	No	Yes	Yes
Register for Events	Yes	Yes	Yes	Yes
Record Attendance	No	No	Yes	Yes
Leave Feedback	Yes	Yes	Yes	Yes
Report Feedback	No	No	Yes	Yes
Delete Reported Feedback	No	No	No	Yes (President/VP only)
Upload Monthly Reports	No	No	Yes (QC only)	Yes
Set Access Permissions	No	No	No	Yes
View Certificates	Yes	Yes	Yes	Yes
Generate Certificates	Yes	Yes	Yes	Yes
Issue Manual Certificates	No	No	Yes	Yes
View Committee Database	Yes	Yes	Yes	Yes
Edit Committee Database	No	No	Yes (own committee)	Yes

4. Pre-Seeded Admin Accounts

These accounts are created by `seed.py` on first run. The frontend team can use them for testing:

Role	Username	Email	Default Password	Committee
President	<code>president</code>	AICHECUSCgeneral@gmail.com	<code>AICHECUSCgeneral@2026</code>	— (global)
Committee Admin	<code>marketing_head</code>	AICHECU.Marketing@gmail.com	<code>AICHECU.Marketing@2026</code>	Marketing
Committee Admin	<code>oc_head</code>	AICHECU.OC@gmail.com	<code>AICHECU.OC@2026</code>	OC
Committee Admin	<code>technical_head</code>	AICHECU.Technical@gmail.com	<code>AICHECU.Technical@2026</code>	Technical
Committee Admin	<code>webdev_head</code>	AICHECU.WebDevelopment@gmail.com	<code>AICHECU.WebDevelopment@2026</code>	Web Development
Committee Admin	<code>pr_head</code>	aichecu.pr@gmail.com	<code>aichecu.pr@2026</code>	PR
Committee Admin	<code>qc_head</code>	aiche.cu.qc@gmail.com	<code>aiche.cu.qc@2026</code>	QC
Committee Admin	<code>general_head</code>	AICHECUSCgeneral@gmail.com	<code>AICHECUSCgeneral@2026</code>	General

[!NOTE]

President and General Committee Head share the same email. Use the **username** field to log in as a specific one.

5. API Endpoints Reference

5.1 Auth

Prefix: `/api/auth`

POST `/api/auth/signup` — Register a new student

Auth: None

Content-Type: `application/json`

Request Body:

```
{
  "full_name": "Ahmed Hassan",
  "username": "ahmed_h",           // 3-50 chars
  "email": "ahmed@example.com",
  "password": "secure123"         // min 6 chars
}
```

Response (201):

```
{
  "id": 15,
  "full_name": "Ahmed Hassan",
  "username": "ahmed_h",
  "email": "ahmed@example.com",
  "role": "student",
  "committee_id": null,
  "committee": null,
  "permissions": null,
  "created_at": "2026-09-18T20:00:00"
}
```

Errors:

- 400 — "Email already registered" or "Username already taken"

[!IMPORTANT]

Signup always creates a `student` role. There is no way for a user to set their own role via the API — this is enforced at the schema level.

POST /api/auth/login — Get JWT token

Auth: None

Content-Type: `application/x-www-form-urlencoded`

Request Body (form data):

```
grant_type=password&username=president&password=AICHECUSCgeneral%402026
```

[!TIP]

The `username` field accepts **both username and email**. If you want to let users login by email, just put the email in the `username` field.

Response (200):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "bearer"
}
```

Errors:

- 401 — "Incorrect username or password"

GET /api/auth/me — Get current user profile

Auth: Bearer Token

Response (200):

```
{
  "id": 1,
  "full_name": "Head President",
  "username": "president",
  "email": "AICHECUSCgeneral@gmail.com",
  "role": "president",
  "committee_id": null,
  "committee": null,
  "permissions": {
    "view_courses": true,
    "add_course": true,
    "edit_course": true,
    "delete_course": true,
    "add_course_part": true,
    "view_workshops": true,
    "add_workshop": true,
    "edit_workshop": true,
    "delete_workshop": true,
    "view_certificates": true
  },
  "created_at": "2026-09-18T20:00:00"
}
```

[!TIP]

Use the `permissions` object from `/me` to conditionally show/hide UI elements (add buttons, edit icons, delete actions). This is the single

source of truth for what the current user can do.

5.2 Dashboard

Prefix: `/api/dashboard`

GET `/api/dashboard/stats` — System statistics

Auth: Bearer Token

Response (200):

```
{
  "students": 42,
  "members": 15,
  "courses": 8,
  "workshops": 3,
  "events": 5,
  "site_visits": 2
}
```

GET `/api/dashboard/recent-activity` — Activity feed

Auth: Bearer Token

Response (200):

```
[
  {
    "id": "session_3",
    "type": "session",
    "message": "Technical Committee added a new session: 'Python Basics'",
    "created_at": "2026-09-18T20:00:00"
  },
  {
    "id": "course_5",
    "type": "course",
    "message": "New course added: 'Chemical Engineering 101'",
    "created_at": null
  },
  {
    "id": "user_12",
    "type": "student",
    "message": "New user signed up: 'Ahmed Hassan'",
    "created_at": "2026-09-18T19:30:00"
  }
]
```

5.3 Courses

Prefix: `/api/courses`

GET `/api/courses` — List all courses

Auth: Bearer Token

Response (200):

```
[
  {
    "id": 1,
    "title": "Chemical Engineering 101",
    "description": "Introduction to ChemE",
    "image_url": "https://example.com/img.jpg",
    "instructor": "Dr. Ahmed",
    "level": "Beginner",
    "duration": "4 weeks",
    "students": 12,
    "parts": [
      {
        "id": 1,
        "course_id": 1,
        "title": "Introduction",
        "type": "lecture",
        "order_index": 1,
        "content_url": "https://youtube.com/..."
      }
    ]
  }
]
```

GET /api/courses/{course_id} — Get single course

Auth: Bearer Token

Response: Same structure as above but single object.

POST /api/courses — Create a course

Auth: Bearer Token

Permission Required: `add_course` (Technical/Web Dev committee heads, President/VP/Secretary)

Request Body:

```
{
  "title": "Chemical Engineering 101",
  "description": "Introduction to ChemE",
  "image_url": "https://example.com/img.jpg",
  "instructor": "Dr. Ahmed",
  "level": "Beginner",
  "duration": "4 weeks"
}
```

Response (201): Course object with `students: 0` and empty `parts: []`

PUT /api/courses/{course_id} — Update a course

Auth: Bearer Token

Permission Required: `edit_course`

Request Body: Same as create (all fields required).

Response (200): Updated course object.

DELETE /api/courses/{course_id} — Delete a course

Auth: Bearer Token

Permission Required: `delete_course`

Response: 204 No Content

[!WARNING]

Deleting a course **cascade-deletes** all its parts and their progress records.

GET /api/courses/{course_id}/parts — List course parts

Auth: Bearer Token

Response (200): Array of `CoursePartOut` objects, ordered by `order_index`.

POST /api/courses/{course_id}/parts — Add a part

Auth: Bearer Token

Permission Required: `add_course_part`

Request Body:

```
{
  "title": "Lecture 1: Introduction",
  "type": "lecture",
  "order_index": 1,
  "content_url": "https://youtube.com/watch?v=..."
}
```

Valid type values: `"lecture"`, `"task"`, `"quiz"`, `"final_exam"`

Response (201): Created part object.

DELETE /api/courses/{course_id}/parts/{part_id} — Delete a part

Auth: Bearer Token

Permission Required: `add_course_part`

Response: 204 No Content

POST /api/courses/course-parts/{part_id}/complete — Complete a course part

Auth: Bearer Token (any logged-in user)

Response (200):

```
{ "message": "Part completed" }
```

Errors:

- 403 — "Previous part not completed yet" (sequential unlock rule)
- 404 — "Course part not found"

[!IMPORTANT]

Sequential Unlock Rule: Part N cannot be completed until Part N-1 is done. Part 1 (order_index=1) is always unlockable. If the part type is "task", completing it automatically awards **+2 points**. Once ALL parts are completed, a certificate is **auto-issued**.

5.4 Workshops

Prefix: /api/workshops

GET /api/workshops — List all workshops

Auth: Bearer Token

Response (200):

```
[
  {
    "id": 1,
    "title": "React Workshop",
    "description": "Learn React fundamentals",
    "image_url": "https://example.com/img.jpg",
    "instructor": "Eng. Sara",
    "sessions_count": 3,
    "sessions": [
      {
        "id": 1,
        "workshop_id": 1,
        "title": "Session 1: Setup",
        "description": "Environment setup",
        "date": "2026-10-01T14:00:00",
        "mode": "online",
        "location_or_link": "https://zoom.us/j/...",
        "materials_url": "https://drive.google.com/..."
      }
    ]
  }
]
```

GET /api/workshops/{workshop_id} — Get single workshop

Auth: Bearer Token

Response: Same structure, single object.

POST /api/workshops — Create a workshop

Auth: Bearer Token

Permission Required: add_workshop

Request Body:

```
{
  "title": "React Workshop",
  "description": "Learn React fundamentals",
  "image_url": "https://example.com/img.jpg",
  "instructor": "Eng. Sara"
}
```

Response (201): Workshop object with sessions_count: 0, sessions: []

PUT /api/workshops/{workshop_id} — Update a workshop

Auth: Bearer Token

Permission Required: edit_workshop

Request Body: Same as create.

Response (200): Updated workshop object.

DELETE /api/workshops/{workshop_id} — Delete a workshop

Auth: Bearer Token

Permission Required: delete_workshop

Response: 204 No Content

[!WARNING]

Deleting a workshop **cascade-deletes** all its sessions.

GET /api/workshops/{workshop_id}/sessions — List workshop sessions

Auth: Bearer Token

Response (200): Array of `WorkshopSessionOut`.

POST /api/workshops/{workshop_id}/sessions — Add a session

Auth: Bearer Token

Permission Required: `add_workshop`

Request Body:

```
{
  "title": "Session 1: Setup",
  "description": "Environment setup & intro",
  "date": "2026-10-01T14:00:00",
  "mode": "online",
  "location_or_link": "https://zoom.us/j/123456",
  "materials_url": "https://drive.google.com/file/..."
}
```

Valid mode values: `"online"`, `"offline"`

Response (201): Created session object.

DELETE /api/workshops/{wid}/sessions/{sid} — Delete a session

Auth: Bearer Token

Permission Required: `edit_workshop`

Response: 204 No Content

5.5 Events & Site Visits

Prefix: `/api/events`

GET /api/events/ — List all events

Auth: No None required (public)

Response (200):

```
[
  {
    "id": 1,
    "type": "event",
    "title": "Annual Conference 2026",
    "description": "Join us for...",
    "date": "2026-11-15T10:00:00+03:00",
    "location": "Cairo University Hall A",
    "registration_deadline": "2026-11-10T23:59:00+03:00",
    "created_by": 1,
    "status": "upcoming"
  }
]
```

[!IMPORTANT]

Event status is computed on-the-fly (UTC+3), never stored:

- `"upcoming"` -> `now < registration_deadline`
- `"closed_registration"` -> `registration_deadline <= now < event date`
- `"finished"` -> `now >= event date`

The frontend should display status badges based on this field and **hide the "Register" button** when status is not `"upcoming"`.

Valid type values: `"event"`, `"site_visit"`

POST /api/events/ — Create an event

Auth: Bearer Token

Roles: `committee_admin`, `president`, `vice_president`, `secretary`

Request Body:

```
{
  "type": "event",
  "title": "Annual Conference 2026",
  "description": "Join us for the biggest event of the year",
  "date": "2026-11-15T10:00:00",
  "location": "Cairo University Hall A",
  "registration_deadline": "2026-11-10T23:59:00"
}
```

POST /api/events/{event_id}/register — Register for event

Auth: Bearer Token (any logged-in user)

Response (200):

```
{ "message": "Registered successfully" }
```

Errors:

- 400 — "Registration is closed" (past deadline)
- 400 — "Already registered"

POST /api/events/{event_id}/attendance — Mark attendance

Auth: Bearer Token

Roles: committee_admin, president, vice_president, secretary

Request Body:

```
{
  "user_id": 15,
  "status": "attended"
}
```

Valid status values: "attended", "absent_excused", "absent_unexcused"

Response (200):

```
{
  "message": "Attendance recorded",
  "points_earned": 2
}
```

5.6 Committee Sessions

Prefix: /api/sessions

GET /api/sessions/ — List visible sessions

Auth: Bearer Token

Roles: member, committee_admin, president, vice_president, secretary

[!IMPORTANT]

Session Visibility Rules:

- **Members / Committee Admins** -> See own committee's sessions + public sessions
- **QC committee** -> Sees ALL committees' sessions (special oversight role)
- **President / VP / Secretary** -> See ALL sessions

Response (200):

```
[
  {
    "id": 1,
    "committee_id": 3,
    "title": "Weekly Meeting #5",
    "description": "Agenda: ...",
    "date": "2026-09-20T18:00:00",
    "visibility": "committee_specific",
    "status": "scheduled",
    "created_by": 5
  }
]
```

Valid visibility values: "public", "committee_specific"

POST /api/sessions/ — Create a session

Auth: Bearer Token

Roles: `committee_admin`, `president`, `vice_president`, `secretary`

Request Body:

```
{
  "title": "Weekly Meeting #6",
  "description": "Review progress on...",
  "date": "2026-09-27T18:00:00",
  "visibility": "committee_specific",
  "status": "scheduled"
}
```

[!NOTE]

`committee_id` and `created_by` are **automatically set from the JWT** — do NOT send them in the body.

GET /api/sessions/{session_id} — Session detail + scoring table

Auth: Bearer Token

Roles: `member`, `committee_admin`, `president`, `vice_president`, `secretary`

Response (200):

```
{
  "id": 1,
  "committee_id": 3,
  "title": "Weekly Meeting #5",
  "description": "...",
  "date": "2026-09-20T18:00:00",
  "visibility": "committee_specific",
  "status": "scheduled",
  "created_by": 5,
  "scoring": [
    {
      "id": 1,
      "session_id": 1,
      "user_id": 10,
      "attendance_status": "on_time",
      "task_status": "on_time",
      "engagement_status": "engaged",
      "bonus_points": 0,
      "recorded_by": 5
    }
  ]
}
```

POST /api/sessions/{session_id}/scoring — Score a member

Auth: Bearer Token

Roles: `committee_admin`, `president`, `vice_president`, `secretary`

Request Body:

```
{
  "user_id": 10,
  "attendance_status": "on_time",
  "task_status": "on_time",
  "engagement_status": "engaged",
  "bonus_points": 1
}
```

Response (200):

```
{
  "message": "Scoring saved",
  "points_added": 6,
  "scoring_id": 1
}
```

[!TIP]

Re-scoring is safe. If you submit scoring for the same `user_id` + `session_id`, it **replaces** the previous scores without duplicating point entries.

5.7 Points

Prefix: `/api/points`

GET /api/points/{user_id}/total — Live total points

Auth: Bearer Token

Roles: member, committee_admin, president, vice_president, secretary

Response (200):

```
{
  "user_id": 10,
  "total_points": 15
}
```

[!IMPORTANT]

Total Points is NEVER stored as an editable column. It is always a live `SUM()` calculation over the `point_entries` table. This means you can never manually set a user's total — it's always computed from individual entries.

GET /api/points/{user_id}/history — Point breakdown

Auth: Bearer Token

Roles: member, committee_admin, president, vice_president, secretary

Response (200):

```
[
  {
    "id": 1,
    "user_id": 10,
    "source_type": "attendance",
    "source_id": 1,
    "points": 2,
    "reason": "Session attendance: on time",
    "created_at": "2026-09-18T20:00:00"
  },
  {
    "id": 2,
    "user_id": 10,
    "source_type": "task",
    "source_id": 1,
    "points": 2,
    "reason": "Task submitted on time",
    "created_at": "2026-09-18T20:00:01"
  }
]
```

source_type values: "attendance", "task", "engagement", "bonus", "event"

Points can be negative (e.g., -1 for unexcused absence).

5.8 Committee Database

Prefix: /api/committees

GET /api/committees/{committee_id}/database — View database rows

Auth: Bearer Token (any logged-in user)

Query Params:

- `skip` (int, default 0) — Pagination offset
- `limit` (int, default 50, max 200) — Page size

Response (200):

```
[
  {
    "id": 1,
    "committee_id": 3,
    "row_data": {
      "member_name": "Ahmed",
      "phone": "01012345678",
      "university_id": "2026-CS-001"
    },
    "created_by": 5,
    "updated_at": "2026-09-18T20:00:00"
  }
]
```

[!TIP]

`row_data` is **flexible JSON** — each committee can define their own columns. The frontend should render this dynamically (e.g., as a table where keys become column headers).

POST /api/committees/{committee_id}/database — Add a row

Auth: Bearer Token

Access: Owning committee members OR users with granted access

Request Body:

```
{
  "row_data": {
    "member_name": "Sara Mohamed",
    "phone": "01098765432",
    "university_id": "2026-CS-042"
  }
}
```

PUT /api/committees/{cid}/database/{row_id} — Edit a row

Auth: Bearer Token

Access: Same as above

Request Body: Same as POST.

Response (200): Updated row.

DELETE /api/committees/{cid}/database/{row_id} — Delete a row

Auth: Bearer Token

Access: Same as above

Response: 204 No Content

5.9 Monthly Reports

Prefix: /api/reports

POST /api/reports/members/{member_id}/monthly-report — Upload report

Auth: Bearer Token

Roles: QC committee admin, President, VP, Secretary

Request Body:

```
{
  "file_url": "https://res.cloudinary.com/.../report_sept.pdf",
  "month": "2026-09"
}
```

[NOTE]

Only the **QC committee** can upload reports (among committee admins). President/VP/Secretary bypass this restriction. The `file_url` should be a pre-uploaded file link (e.g., from Cloudinary).

GET /api/reports/members/{member_id} — List member's reports

Auth: Bearer Token

Roles: member, committee_admin, president, vice_president, secretary

Response (200):

```
[
  {
    "id": 1,
    "member_id": 10,
    "file_url": "https://...",
    "month": "2026-09",
    "uploaded_by": 7,
    "uploaded_at": "2026-09-18T20:00:00"
  }
]
```

5.10 Feedback & Moderation

Prefix: /api/feedback

POST /api/feedback/ — Leave feedback

Auth: Bearer Token (any logged-in user)

Request Body:

```
{
  "target_type": "course",
  "target_id": 1,
  "comment": "Great course, very helpful!"
}
```

Valid **target_type** values: "course", "workshop", "event", "site_visit"

GET /api/feedback/ — List feedback

Auth: No None required (public)

Query Params (optional):

- **target_type** — Filter by type
- **target_id** — Filter by specific item

Response (200): Array of feedback objects (only non-deleted).

GET /api/feedback/reported — View moderation queue

Auth: Bearer Token

Roles: president, vice_president, secretary

Response (200): Array of reported (flagged) but not-yet-deleted feedback.

POST /api/feedback/{feedback_id}/report — Flag for review

Auth: Bearer Token

Roles: committee_admin, president, vice_president, secretary

Response (200):

```
{
  "message": "Feedback reported",
  "is_reported": true
}
```

DELETE /api/feedback/{feedback_id} — Approve deletion

Auth: Bearer Token

Roles: president, vice_president **ONLY**

Response (200):

```
{ "message": "Comment and account deleted" }
```

[!CAUTION]

This is a destructive action. Approving deletion removes the comment AND **permanently deletes the author's entire account** along with all their points, progress, registrations, and scoring records. This is the "Rule 8" nuclear option.

5.11 Access Permissions

Prefix: /api/access-permissions

POST /api/access-permissions/ — Grant/revoke access

Auth: Bearer Token

Roles: president, vice_president, secretary

Request Body:

```
{
  "section_name": "courses",
  "committee_id": 3,
  "can_edit": true
}
```

[!NOTE]

Uses **upsert logic**: if a permission row already exists for the same (**section_name**, **committee_id**), it updates instead of duplicating. Calling this twice with the same data just toggles **can_edit**.

GET /api/access-permissions/ — List all permissions

Auth: Bearer Token

Roles: `president`, `vice_president`, `secretary`

Response (200):

```
[
  {
    "id": 1,
    "section_name": "courses",
    "committee_id": 3,
    "can_edit": true,
    "granted_by": 1,
    "updated_at": "2026-09-18T20:00:00"
  }
]
```

5.12 Certificates

Prefix: `/api/certificates`

GET `/api/certificates` — List my certificates

Auth: Bearer Token

Response (200):

```
[
  {
    "id": 1,
    "user_id": 10,
    "type": "course",
    "source_id": 1,
    "pdf_url": "https://res.cloudinary.com/.../cert.png",
    "issued_at": "2026-09-18T20:00:00"
  }
]
```

GET `/api/certificates/{certificate_id}` — Get certificate details

Auth: Bearer Token

Response: Single certificate object.

POST `/api/certificates/generate` — Generate certificate

Auth: Bearer Token

Request Body:

```
{
  "source_type": "course",
  "source_id": 1
}
```

Valid `source_type` values: `"course"`, `"workshop"`, `"event"`, `"session"`

[!IMPORTANT]

Certificate generation **validates completion first**. The user must have actually completed the course / attended the workshop / attended the event / attended the session. Otherwise it returns **400**.

POST `/api/certificates/manual` — Issue custom certificate

Auth: Bearer Token

Roles: `committee_admin`, `president`, `vice_president`, `secretary`

Request Body:

```
{
  "username": "ahmed_h",
  "title": "Outstanding Performance Award"
}
```

Response (201): Certificate object with the generated PNG URL.

6. Business Logic Rules

Rule 1: Sequential Course Unlock

- Course parts MUST be completed in order (by `order_index`)
- Part 1 is always available
- Part N requires Part N-1 to be completed first
- Violation returns `403 Forbidden`

Rule 2: Live Point Calculation

- Total points = `SUM(point_entries.points)` — always computed, never stored
- Points can be **negative** (deductions for absences)
- Frontend should display the total from `/api/points/{user_id}/total`

Rule 3: Event Status is Computed (UTC+3)

- Never stored in the database
- Calculated at response time based on `date` and `registration_deadline`
- Frontend should use the `status` field from the API response directly

Rule 4: Session Visibility

- Members see: own committee sessions + public sessions
- QC: sees ALL sessions (special oversight)
- President/VP/Secretary: sees ALL sessions

Rule 5: Feedback Moderation (Two-Step)

1. **Step 1:** OC/Admin flags a comment -> adds to review queue
2. **Step 2:** President/VP approves deletion -> removes comment AND author's account

Rule 6: Re-Scoring Safety

- Submitting scoring for the same user+session replaces previous scores
- Old point entries are deleted before new ones are created
- No duplicate points possible

Rule 7: Auto-Certificate on Course Completion

- When all parts of a course are marked completed, a certificate is automatically generated
- The certificate is uploaded to cloud storage and the URL is saved
- Idempotent: calling it again returns the existing certificate

Rule 8: Cascade Deletions

- Deleting a **Course** -> deletes all parts and progress records
- Deleting a **Workshop** -> deletes all sessions
- Deleting a **Feedback** (approval) -> deletes author's account + all their data

7. Point System Scoring Tables

Session Attendance Points

Status	Value to Send	Points	Reason
On time	<code>"on_time"</code>	+2	"Session attendance: on time"
Late (< 15 min)	<code>"late_under_15"</code>	+1	"Session attendance: late (under 15 min)"
Late (> 15 min)	<code>"late_over_15"</code>	0	"Session attendance: late (over 15 min)"
Absent (excused)	<code>"absent_excused"</code>	0	"Session attendance: absent (excused)"

Status	Value to Send	Points	Reason
Absent (unexcused)	"absent_unexcused"	-1	"Session attendance: absent (unexcused)"

Session Task Points

Status	Value to Send	Points	Reason
Submitted on time	"on_time"	+2	"Task submitted on time"
Submitted late	"late"	0	"Task submitted after deadline"
Not submitted	"not_submitted"	-1	"Task not submitted"

Session Engagement Points

Status	Value to Send	Points	Reason
Engaged	"engaged"	+1	"Engaged during session"
Not engaged	"not_engaged"	0	"Not engaged during session"

Bonus Points

- Manually entered by admin as any integer
- Only recorded if non-zero

Event Attendance Points

Status	Value to Send	Points	Reason
Attended	"attended"	+2	"Event attended"
Absent (excused)	"absent_excused"	0	"Event absent (excused)"
Absent (unexcused)	"absent_unexcused"	-1	"Event absent without excuse"

Course Task Points

- Completing a part of type "task" -> **+2 points** ("Task submitted on time")

8. Error Handling Guide

Standard HTTP Error Codes

Code	Meaning	When It Happens
400	Bad Request	Invalid input, duplicate email/username, registration closed, already registered
401	Unauthorized	Missing/invalid/expired token
403	Forbidden	User lacks permission for this action
404	Not Found	Resource doesn't exist
422	Validation Error	Missing required fields or wrong data types
500	Internal Error	Server-side error (rare, report if seen)

Error Response Format

```
{
  "detail": "Human-readable error message"
}
```

For 422 validation errors:

```
{
  "detail": [
```

```

    {
      "loc": ["body", "title"],
      "msg": "field required",
      "type": "value_error.missing"
    }
  ]
}

```

[!TIP]

Always check for **401** globally in your API client. If received, redirect to the login page and clear the stored token.

9. Frontend Tips & Best Practices

1. Use `/api/auth/me` as the Single Source of Truth

After login, call `GET /api/auth/me` and store the response. Use:

- `role` -> for route guarding (which pages to show)
- `permissions` -> for element-level visibility (show/hide buttons)
- `committee_id` -> for committee-specific views
- `committee` -> for display name

2. Handle Token Lifecycle

```

// Store token after login
localStorage.setItem('token', data.access_token);

// Create an axios interceptor
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Handle 401 globally
api.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

```

3. Conditional UI Rendering

```

// After fetching user from /me
const user = await api.get('/api/auth/me');

// Show "Add Course" button only if permission allows
if (user.permissions?.add_course) {
  showAddCourseButton();
}

// Show "Delete Feedback" only for president/vice_president
if (['president', 'vice_president'].includes(user.role)) {
  showFeedbackModerationPanel();
}

// Show "Upload Report" only for QC
if (user.role === 'committee_admin' && user.committee === 'QC') {
  showUploadReportButton();
}

```

4. Login Form Implementation

```

// IMPORTANT: Login uses form data, NOT JSON
async function login(username, password) {
  const formData = new URLSearchParams();
  formData.append('grant_type', 'password');
  formData.append('username', username); // accepts username OR email
  formData.append('password', password);

  const response = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
    body: formData.toString()
  });
}

```

```
const data = await response.json();
return data.access_token;
}
```

5. Event Status Display

```
// Status comes pre-computed from the API
function renderEventBadge(event) {
  switch (event.status) {
    case 'upcoming':
      return <Badge color="green">Open for Registration</Badge>;
    case 'closed_registration':
      return <Badge color="yellow">Registration Closed</Badge>;
    case 'finished':
      return <Badge color="gray">Finished</Badge>;
  }
}

// Show "Register" button only for upcoming events
{event.status === 'upcoming' && <RegisterButton eventId={event.id} />}
```

6. Scoring Form UI

Build a form with dropdowns for each scoring dimension:

```
+-----+
| Score Member: [Dropdown: Select Member] |
| Attendance: [on_time v]                 |
| Task: [on_time v]                       |
| Engagement: [engaged v]                 |
| Bonus: [0_____]                         |
| [Submit Scoring]                       |
+-----+
```

7. Course Progress UI

```
Course: Chemical Engineering 101
+-- Yes Part 1: Introduction (lecture) – Completed
+-- Yes Part 2: Fundamentals (lecture) – Completed
+-- [Available] Part 3: Lab Task (task) – Available <- user can open this
+-- [Locked] Part 4: Quiz (quiz) – Locked <- greyed out
+-- [Locked] Part 5: Final Exam (final_exam) – Locked
```

8. Committee Database Dynamic Rendering

Since `row_data` is flexible JSON, render it dynamically:

```
// Get unique keys from all rows to build column headers
const allKeys = new Set();
rows.forEach(row => Object.keys(row.row_data).forEach(k => allKeys.add(k)));
const columns = Array.from(allKeys);

// Render as table
<table>
  <thead>
    <tr>{columns.map(col => <th>{col}</th>)}</tr>
  </thead>
  <tbody>
    {rows.map(row => (
      <tr>{columns.map(col => <td>{row.row_data[col] || '-'}</td>)}</tr>
    ))}
  </tbody>
</table>
```

9. Point History Display

Show the breakdown with color coding:

```
Points History for Ahmed Hassan (Total: 15 pts)
-----
+2 * Session attendance: on time           Sep 18
+2 * Task submitted on time                Sep 18
+1 * Engaged during session               Sep 18
+2 * Event attended                       Sep 15
-1 * Task not submitted                   Sep 10
```

10. File Upload Flow (for Reports & Certificates)

1. Upload the file to **Cloudinary** (or your preferred storage) from the frontend
2. Get the returned URL

3. Send the URL to the backend API (`file_url` field)
4. The backend stores the URL, not the file itself

[!TIP]

Swagger UI at `{BASE_URL}/docs` lets you test every endpoint interactively — use the "Authorize" button ([Locked]) at the top to paste a JWT token, then try out any endpoint directly.